Hockey UQ Project — Architecture & Build Plan

A probabilistic hockey prediction pipeline with full uncertainty quantification, structured as a port and extension of the AEM 8290 land-surface column UQ project. The goal is calibrated predictive distributions over game outcomes, player point totals, and (eventually) season-long quantities — not point estimates.

This document is the project's reference architecture and roadmap. Come back to it when you forget what `analysis/` is supposed to contain or why `game_model.py` lives at the top level of `models/`.

Project Goals

The end-state pipeline produces, for any matchup:

- $P(\text{home win})$ with credible intervals
- Predictive distributions over final score, total goals, goal differential
- Predictive distributions over individual player point totals
- Sensitivity decomposition: which inputs drive outcome variance

All of these come from the same underlying machinery — a hierarchical Bayesian model fit to league-wide data, propagated through a game model and Monte Carlo simulator, validated with proper scoring rules.

First shipping target: single-game win probability. Smallest pipeline that exercises every layer.

Status

Last updated: 2026-10-14

Phase	Status	Result
1. Data foundation	**Done**	5 seasons (2021-22 to 2025-26), 6,560 games, full play-by-play, rebound/rush context validated
2. xG model	**Done**	LightGBM, test AUC 0.769, team-season xG correlates 0.97 with MoneyPuck
3. Team strength & goalie aggregates	**Next**	
4–8	Not started	

Check current state: `py .\scripts\check_state.py` lists which files and tables exist and whether `shot_context` matches the current derivation code.

The Six-Step Arc

The full pipeline, end-to-end:

1. **Build component models** — xG (gradient boosting), team strength aggregates, goalie ratings. Each produces estimates with uncertainty.
2. **Combine via Poisson/NB game model** — maps team-level inputs to $\lambda_{\text{home}}, \lambda_{\text{away}}$.
3. **Monte Carlo simulation** — propagates uncertainty from every layer into a predictive distribution over outcomes.
4. **Aggregate sims** into the distributions you actually care about: win probabilities, score distributions, player point totals.
5. **Validate** with proper scoring rules (log-loss, Brier, CRPS) and calibration plots on held-out games.
6. **Fit PCE surrogate** over the simulator's input space — enables fast sensitivity analysis and analytical Sobol decomposition.

Mapping to the AEM 8290 Soil-Atmosphere Project

The hockey UQ pipeline is structurally the same as your AEM 8290 land-surface column project, with the forward model swapped and a hierarchical layer added. Most of the conceptual machinery already exists in working code.

Hockey project component	Soil project equivalent
Component models (xG, team strength, goalie)	Forward model: soil heat + Richards' + SEB
Game model ($\lambda_{\text{home}}, \lambda_{\text{away}}$)	Forward model mapping $\theta \rightarrow$ QoIs
Monte Carlo over game outcomes	Forward propagation, screening, Sobol Monte Carlo
Aggregation to predictive distributions	Posterior moments, predictive distributions
Held-out validation, calibration plots	LOO-CV Q^2, held-out RMSE, trace plots, ACF
PCE surrogate for Sobol indices	PCE surrogate for MCMC efficiency

What transfers directly:

- The Adaptive Metropolis sampler (random-walk MH, burn-in / adapt / sample, $(2.38)^2/d$ scaling, $\xi \in [-1, 1]^d$ standardized space)
- Additive Gaussian likelihood structure (mechanics — independent QoI noise summed in log-space)
- Objective prior strategy (uniform-bounded for location, log-uniform for scale spanning decades)
- PCE-as-surrogate trick for expensive forward models
- Validation discipline (Q^2, trace plots, acceptance rates, ACF, posterior recovery vs truth)

What changes:

- **Dimensionality.** Soil: 5 calibration parameters. Hockey hierarchical model: ~70–100+ parameters. Random-walk MH does not scale; NUTS (HMC variant via PyMC/NumPyro) required.
- **Hierarchy.** Soil model is flat. Hockey model has population-level hyperpriors with partial pooling across teams and positions.
- **Likelihood form.** Gaussian becomes Poisson (goal counts) or Bernoulli (win/loss).
- **Forward model speed.** Soil: ~1 s/eval, surrogate essential. Hockey game model: milliseconds/eval, surrogate optional. PCE earns its keep in hockey for the **Sobol analysis**, not for inference speed.
- **Data volume.** Soil: synthetic 24–72h time series of 2 QoIs. Hockey: ~1200 games/season × multiple observables per game.

Architecture

Folder Structure

```
hockey-stats/
│
├── pyproject.toml
├── README.md
├── .gitignore
├── .env.example
│
├── config/
│   ├── settings.py            # paths, current season, DB location
│   └── teams.py               # team metadata, colors, abbreviations
│
├── data/                      # SQLite database lives here
│   ├── hockey_stats.db
│   └── external/              # files downloaded by hand (e.g. MoneyPuck teams.csv)
│
├── src/
│   │
│   ├── data/                  # data acquisition & storage
│   │   ├── __init__.py
│   │   ├── nhl_api.py         # NHL API client
│   │   ├── db.py              # SQLite operations
│   │   ├── scraper.py         # Natural Stat Trick / Evolving-Hockey scrapers
│   │   └── cache.py           # caching layer
│   │
│   ├── processing/            # raw data → analysis-ready
│   │   ├── __init__.py
│   │   ├── shot_context.py    # DONE: rebound / rush flags → shot_context table
│   │   ├── xg_processing.py   # DONE: xG feature table, one row per shot
│   │   ├── shots.py           # shot data processing
│   │   ├── corsi.py           # Corsi/Fenwick calculations
│   │   ├── game_score.py      # game score breakdown
│   │   ├── team_strength.py   # rolling xGF/60, xGA/60 aggregates
│   │   ├── rapm.py            # RAPM / WAR (future)
│   │   └── goalie.py          # goalie statistics
│   │
│   ├── models/
│   │   ├── __init__.py
│   │   ├── schemas.py         # Pydantic: Game, Player, Shot, ScheduleGame
│   │   ├── game_model.py      # TOP-LEVEL: maps team inputs → λ_home, λ_away
│   │   │
│   │   ├── deterministic/
│   │   │   ├── __init__.py
│   │   │   ├── xg.py          # DONE: ExpectedGoalsModel (LightGBM)
│   │   │   └── rapm.py        # ridge regression RAPM (future)
│   │   │
│   │   └── stochastic/
```

```
│   │       ├── __init__.py
│   │       ├── am_sampler.py   # Adaptive Metropolis (ported from soil)
│   │       ├── goalie.py       # Bayesian goalie rating (simple conjugate)
│   │       ├── toy_mcmc.py     # Track A: hockey-flavored AM MCMC
│   │       ├── hierarchical.py # Track B: PyMC hierarchical model
│   │       ├── montecarlo.py   # Monte Carlo over posterior samples
│   │       └── pce.py          # PCE surrogate for Sobol analysis
│   │
│   ├── analysis/               # NEW: reduce sim output to distributions
│   │   ├── __init__.py
│   │   ├── aggregation.py      # P(win), credible intervals, marginals
│   │   └── sensitivity.py      # Sobol indices from PCE coefficients
│   │
│   ├── validation/             # NEW: scoring, calibration, backtesting
│   │   ├── __init__.py
│   │   ├── scoring.py          # log-loss, Brier, CRPS
│   │   ├── calibration.py      # reliability diagrams
│   │   └── backtest.py         # walk-forward validation harness
│   │
│   ├── pipelines/              # NEW: high-level orchestration
│   │   ├── __init__.py
│   │   ├── train_xg.py         # DONE: build features, train, evaluate, save
│   │   ├── fit_hierarchical.py # fit PyMC model, save ArviZ NetCDF
│   │   ├── fit_toy.py          # fit Track A toy MCMC for comparison
│   │   ├── predict_game.py     # end-to-end single-game prediction
│   │   ├── backtest_season.py  # run validation harness on held-out games
│   │   └── build_pce.py        # fit PCE surrogate for Sobol
│   │
│   ├── viz/                    # visualization (existing)
│   │   ├── __init__.py
│   │   ├── theme.py
│   │   ├── rink_plot.py        # DONE: draw_rink, plot_game_events, plot_game
│   │   ├── shot_heatmap.py
│   │   ├── game_summary.py     # (future)
│   │   ├── player_card.py      # (future)
│   │   └── components/
│   │
│   └── export/
│       ├── __init__.py
│       └── save.py
│
├── notebooks/                  # exploration and checks
│   ├── data_sanity.ipynb       # raw data checks (sections 1-13)
│   ├── xg_features_check.ipynb # xG feature table checks
│   └── xg_model_dev.ipynb      # baseline, LightGBM, evaluation, MoneyPuck comparison
│
├── output/                     # generated images, posterior samples
│   ├── game_summaries/
│   ├── player_cards/
│   ├── posteriors/             # NetCDF posterior samples
│   └── models/                 # pickled trained models (xG); keep out of git
│
├── tests/
│   ├── test_corsi.py
│   ├── test_game_score.py
│   └── fixtures/
│
└── scripts/                    # CLI entry points
    ├── load_all.py                   # teams, rosters, schedule, games, shots, events, shifts
    ├── backfill_game_events.py       # one-time: full play-by-play for loaded games
    ├── rebuild_shot_context.py       # re-derive shot_context after a code change
    ├── check_state.py                # project state + shot_context sync check
    ├── fix_goal_shooters.py          # one-time repair (goal scorer IDs)
    ├── check_block_attribution.py    # diagnostic: who owns a blocked-shot event
    ├── check_block_zone_frame.py     # diagnostic: which frame blocked-shot zones use
    ├── nst_spotcheck.py              # compare rush/rebound counts with Natural Stat Trick
    └── rush_event_audit.py           # list rush-flagged shots for one game
```

Why Certain Things Live Where They Do

`models/game_model.py` at the top level of `models/` — neither purely deterministic nor purely stochastic. It's the bridge that takes deterministic components and lifts them into a stochastic combiner. Forcing it into one subfolder would be a category error. It belongs alongside `schemas.py` as a top-level model concern.

`analysis/` separate from `models/` — `models/` is about computation that produces parameters or predictions. `analysis/` is about *reducing* simulation output to the distributions you care about. Different responsibility: aggregation, not modeling.

`validation/` separate from everything — scoring rules and calibration plots are infrastructure that applies to any model. Keeping them separate means they can be reused if you swap component models.

`pipelines/` for orchestration — high-level entry points that chain components together. Each pipeline file should read like a recipe: load this, fit that, save the output. No business logic, just glue.

Two tracks under `stochastic/` — `toy_mcmc.py` (Track A, your soil-pipeline port) and `hierarchical.py` (Track B, PyMC) coexist. Track A is the reference implementation; Track B is the production model. They share input contracts so the rest of the pipeline doesn't care which produced the samples.

Data Contract

Tables

Table	Rows	One row per	Notes
`shot_events`	773,259	shot attempt (incl. blocked, shootout)	Raw; filter at query time (below)
`game_events`	2,060,996	play-by-play event of any type	Raw; ordered by `sort_order`, never `event_id`
`shot_context`	566,545	unblocked, non-shootout shot	Derived from `game_events` by `shot_context.py`
`shifts`	4,578,542	player shift	
`games`, `schedule`	6,560	game	1,312 per season × 5

`shot_context` joins to `shot_events` on `(game_id, event_id)`. After any change to `src/processing/shot_context.py`, run `scripts/rebuild_shot_context.py`; `check_state.py` flags a stale table.

Filtering Rules

`shot_events` is stored **raw and complete** — no filtering is applied at the database level. Shootout attempts (period 5) and blocked shots are all present. This is deliberate: the raw table stays a faithful

record, and every consumer applies the filters it needs at query time.

Two rules, applied at query time:

1. `period <= 4` — excludes shootout (period 5) events; keeps OT (period 4).
2. `event_type IN ('shot-on-goal', 'goal', 'missed-shot')` — excludes blocked shots (degenerate coordinates) and shootout attempts.

Together these yield **566,545 clean unblocked non-shootout shots**. The xG feature table then drops empty-net shots (opposing goalie digit = 0 in `situation_code`), leaving **562,374 training shots** (Phase 2).

Why this matters per consumer:

- **xG model (Phase 2):** apply both rules, then drop empty-net shots. Blocked shots lack a true shot location, shootout attempts are mislabeled as goals, and empty-net shots score regardless of geometry; any of them would poison training.
- **Game model (Phase 6):** use `period <= 4` goals as the Poisson target so shootout-winner credits don't inflate goal counts. The shootout is modeled separately (flat 50/50 conditional on a tied regulation — see deferred OT/SO decision).

Background: the data-sanity notebook found that period-5 shootout attempts are stored as `event_type` `'goal'` / `'shot-on-goal'` / `'missed-shot'` (1,065 shootout "goals", ~2.6% of all goal events). All score-reconciliation residuals after filtering are off-by-exactly-1 (the shootout-winner credit added to the official score with no corresponding real goal event), zero off-by-more-than-1 — fully characterized and expected, not a bug.

Known Data Quirks (verified against this database)

- **Goal scorers:** the API stores the scorer under `scoringPlayerId`, not `shootingPlayerId`. `parse_shot_events` reads both. 41,536 goals were repaired once with `scripts/fix_goal_shooters.py`.
- **Blocked-shot owner:** `eventOwnerTeamId` is the **shooting** team (90/10, `check_block_attribution.py`).
- **Blocked-shot zone:** `zoneCode` is written from the **defending** team's side even though the owner is the shooter (99.9% of team-periods, `check_block_zone_frame.py`). `shot_context.py` flips it before the rush rule.
- **Zone codes** are relative to the event owner (O / N / D). Compare two events only after translating both into the shooter's frame.
- **`situation_code`** is 4 digits: away goalie, away skaters, home skaters, home goalie (`1551` = 5v5). Decode from the shooter's point of view.
- **Shot-type labels shifted in 2025-26** (many wrist → snap, deflected → tip-in). The xG features group them as `wrist-snap` and `redirect`.
- **Missing shot type** occurs almost only on goals; filling it with "unknown" leaks the outcome. The xG features fill it with "wrist".
- **Coordinates:** center ice is (0, 0), goal lines at x = ±89, blue lines at x = ±25. Teams switch ends each period; normalize per game, period and team.

Rebound and Rush Definitions (`shot_context.py`)

NST conventions, with the no-stoppage rule enforced (faceoffs, whistles, penalties and goals break the chain):

- **Rebound:** unblocked shot within 3 s of a prior attempt by the same team. 9.4% of shots; converts at 12.4% vs 6.6% for other shots.
- **Rush:** unblocked shot within 4 s of any event in the neutral zone or the shooter's defensive zone. 2.7% of shots; weak as a yes/no flag (7.4% vs 7.1%), but strong when split by the previous event: takeaway 10.1%, blocked shot 11.1%, giveaway 9.0%, hit 4.0%.
- NST spot-check: 1 of 3 rush flags matched in game 2025020993. Both misses had an opponent's shot as the precursor, at physically impossible 1–2 s gaps (about 17% of rush flags, 0.5% of training data).

Parallel Build: Track A vs Track B

The honest version of the model is the PyMC hierarchical NUTS sampler. But porting your AEM 8290 AM MCMC structure to a tiny hockey problem first buys real value: a reference implementation you trust, and an independent cross-check on the PyMC model.

Track A — Toy AM MCMC (Soil Pipeline Port)

Scope: one team pair at a time. Parameter vector:

$$\theta = \left(\mu_{\text{off,A}}, \mu_{\text{def,A}}, \mu_{\text{off,B}}, \mu_{\text{def,B}} \right)$$

Likelihood: Poisson on observed goal counts in head-to-head games:

$$p\left(y^{\text{obs}} \mid \theta \right) = \prod_{g} \text{Poisson}\left(G_{\text{home},g} \mid \lambda_{\text{home}}\left(\theta \right) \right) \cdot \text{Poisson}\left(G_{\text{away},g} \mid \lambda_{\text{away}}\left(\theta \right) \right)$$

Priors: bounded uniform in standardized $\xi \in \left[-1, 1 \right]^{4}$ space, same as soil.

Sampler: Adaptive Metropolis, three phases:

1. **Burn-in** ($n_b = 5000$): isotropic proposal $\Sigma_p = \sigma_0^2 I$ with $\sigma_0 = 0.1$
2. **Adapt:** estimate posterior covariance from burn-in, set $\Sigma_p = s \cdot \widehat{\Sigma}$ with $s = \left(2.38 \right)^2 / d$
3. **Sample:** main run with fixed adapted covariance

Reuses: your soil sampler module, standardization machinery, diagnostics (trace plots, ACF, acceptance rate, R-hat).

Purpose: proof of life, reference posteriors, bug-catching for Track B.

Track B — Hierarchical PyMC Model

Scope: full league simultaneously.

Model:

$$
\begin{aligned}
\mu_{\text{off},\,t} &\sim \mathcal{N}\left(0, \sigma_{\text{off}}^2\right) \quad \text{for each team } t \\
\mu_{\text{def},\,t} &\sim \mathcal{N}\left(0, \sigma_{\text{def}}^2\right) \quad \text{for each team } t \\
\gamma_{\text{goalie},\,g} &\sim \mathcal{N}\left(0, \sigma_{\text{goalie}}^2\right) \quad \text{for each goalie } g \\
\alpha_{\text{home}} &\sim \mathcal{N}\left(0, 1\right) \\
\log \lambda_{\text{home}} &= \alpha_{\text{home}} + \mu_{\text{off},\,h} - \mu_{\text{def},\,a} - \gamma_{\text{goalie},\,a} \\
\log \lambda_{\text{away}} &= \mu_{\text{off},\,a} - \mu_{\text{def},\,h} - \gamma_{\text{goalie},\,h} \\
G_{\text{home}} &\sim \text{Poisson}\left(\lambda_{\text{home}}\right) \\
G_{\text{away}} &\sim \text{Poisson}\left(\lambda_{\text{away}}\right)
\end{aligned}
$$

Hyperpriors on $\sigma_{\text{off}}, \sigma_{\text{def}}, \sigma_{\text{goalie}}$ — typically half-normal or half-Cauchy.

Sampler: NUTS, default PyMC settings to start, reparameterize if divergences appear.

Output: ArviZ `InferenceData` with full joint posterior across all teams and goalies.

Purpose: the actual production model.

Cross-Validation Between Tracks

For any team pair (A, B):

1. Fit Track A on (A, B) head-to-head data only.
2. Marginalize Track B's joint posterior down to the four parameters $\left(\mu_{\text{off},\,A}, \mu_{\text{def},\,A}, \mu_{\text{off},\,B}, \mu_{\text{def},\,B}\right)$.
3. Compare marginal posteriors. They should be close but not identical — Track B benefits from partial pooling across all teams, Track A doesn't.

When they disagree wildly, one of them is wrong, and you have a much better debugging position than with just one implementation.

Build Sequence

Realistic for a side project with focused sessions a few times a week. Roughly two months to a usable single-game predictor.

Phase 1 — Data Foundation (Weeks 1–2) — DONE

- ~~Expand `scripts/load_shots.py` to iterate over the full league schedule.~~ Done as `scripts/load_all.py`: all 32 teams, 5 seasons.
- ~~Verify shot data integrity.~~ `notebooks/data_sanity.ipynb`, sections 1–13, all passing.
- ~~Backfill 2–3 seasons.~~ 2021-22 through 2025-26 (6,560 games).

Added beyond the original plan:

- `game_events` table (full play-by-play, every event type), needed for rush detection.
- `shot_context` table with rebound / rush flags, prior event type and time since prior event.

- Fixes found by the checks: goal-shooter repair, blocked-shot zone frame, no-stoppage rule (see Data Contract → Known Data Quirks).
- External check against Natural Stat Trick (`scripts/nst_spotcheck.py`, `scripts/rush_event_audit.py`).

Phase 2 — xG Model (Week 3) — DONE

Files:

- `src/processing/xg_processing.py` — `build_xg_features(conn)`: one row per shot, 13 features
- `src/models/deterministic/xg.py` — `ExpectedGoalsModel` with `.fit()`, `.predict()`, `.evaluate()`, `.calibration()`, `.feature_importance()`, `.save()`, `.load()`
- `src/pipelines/train_xg.py` — run with `py -m src.pipelines.train_xg [--final]`
- `notebooks/xg_features_check.ipynb`, `notebooks/xg_model_dev.ipynb`

Features (13): distance, angle, shot type (grouped), rebound flag, rush flag, previous event type, time since previous event, skater difference, total skaters, own goalie pulled, distance from previous event, speed from previous event, rebound angle change.

Model: LightGBM, log-loss objective. `num_leaves=31`, `min_child_samples=200`, `learning_rate=0.03`, `subsample=0.8`, `colsample_bytree=0.8`. A 9-combination grid search found no better setting (top six within 0.00005 log loss; 63 leaves worse).

Season roles: train 2021-22 to 2023-24, validate 2024-25 (early stopping picks 387 trees), then refit on 2021-22 to 2024-25 and test once on 2025-26. Recency weights 1.0 / 1.25 / 1.5 / 2.0, oldest to newest.

Results on the 2025-26 test season:

	Baseline (distance + angle, logistic)	Final LightGBM
AUC	0.714	**0.769**
Log loss	0.2307	**0.2168**
Predicted vs actual goals	+1.4%	+2.4% (7,763 vs 7,578)
Calibration	Mid-range under, top over	Within 0.8 points in every decile

What moved the score, in order: LightGBM over logistic regression; previous-event features; recency weighting; shot-type grouping (fixed the goal total).

MoneyPuck comparison (2025-26, 5-on-5, team-season totals, 32 teams): xGF r = 0.970, xGA r = 0.940, total xG ratio 1.000. MoneyPuck blocks scripted downloads; save `teams.csv` by hand to `data/external/`.

Saved models (`output/models/`):

- `xg_lgbm_20212022_20242025.pkl` + `_metrics.json` — the evaluated model
- `xg_lgbm_live.pkl` — same settings, trained on all five seasons, for scoring new games

Gaps against the original plan:

- MoneyPuck validation was done on team-season totals, not per-game totals. Per-game is stricter and still to do.
- No version number in the file name and no config entry pointing to the active model.
- Random seed and library versions are not yet saved with the model.
- Method is named `.predict()`, not `.predict_proba()`.

Phase 3 — Team Strength & Goalie Aggregates (Week 4) — NEXT

- **First:** score every shot with the xG model and store the result in a `shot_xg` table. Team xGF/60 and goalie GSAx are sums over that table. Score each season with a model that did not train on it (walk-forward), or past teams' xG is flattered by a model that has seen their games. `shot_events` already stores `goalie_id` for GSAx.
- `src/processing/team_strength.py`: rolling 25-game windows of xGF/60 and xGA/60 per team.
- `src/models/stochastic/goalie.py`: simple Bayesian goalie rating. Beta-Binomial on save proportion, or Normal-Normal on GSAx/60. No hierarchy yet — that comes in Track B.
- Both produce: point estimate + variance for each team / goalie at any point in the season.

Phase 4 — Track A Toy MCMC (Week 5)

- Port your AM sampler from soil to `src/models/stochastic/am_sampler.py`.
- Write `src/models/stochastic/toy_mcmc.py`: 4-parameter Poisson model for one team pair.
- Write `src/pipelines/fit_toy.py`: load team pair data, run AM, save posterior samples (ArviZ NetCDF for consistency with Track B), produce diagnostics (trace plots, ACF, acceptance rate).
- Validate: synthetic recovery on simulated data with known θ^*, like the soil project's posterior recovery check.

Phase 5 — Track B Hierarchical PyMC (Week 6)

- Write `src/models/stochastic/hierarchical.py` with the full league model.
- Write `src/pipelines/fit_hierarchical.py`: load full-season game data, fit, save InferenceData.
- Diagnostics: R-hat, ESS, divergences, energy plots.
- Cross-validate against Track A on 2–3 team pairs.

Phase 6 — Game Model + Monte Carlo (Week 7)

- Write `src/models/game_model.py`: function taking $\left(\mu_{\text{off}}, \mu_{\text{def}}, \gamma_{\text{goalie}}, \text{home flag}\right)$ samples and producing $\left(\lambda_{\text{home}}, \lambda_{\text{away}}\right)$ samples.
- Write `src/models/stochastic/montecarlo.py`: draws posterior samples, runs game model, samples Poisson goal counts, handles OT/SO.
- Write `src/pipelines/predict_game.py`: end-to-end orchestration. Takes two teams + date, returns predictive distribution.
- Write `src/analysis/aggregation.py`: reduce sim output to $P(\text{ win })$, credible intervals on score margin, etc.

Phase 7 — Validation (Week 8)

- `src/validation/scoring.py`: log-loss, Brier score, CRPS for goal totals.

- `src/validation/calibration.py`: reliability diagrams with bootstrap confidence bands.
- `src/validation/backtest.py`: walk-forward harness. For each held-out game in chronological order, fit components only on prior data, predict, score.
- `src/pipelines/backtest_season.py`: run the full validation on a held-out season tail.

Phase 8 — PCE for Sobol (Week 9+)

- `src/models/stochastic/pce.py`: port PCE machinery from soil. Legendre basis (uniform inputs), total-order truncation, non-intrusive regression.
- Fit on team-level input ranges (e.g., xGF/60 ∈ [2.0, 3.5], goalie GSAx ∈ [-0.3, 0.5]) with $P(\text{ home win })$ as the QoI.
- `src/analysis/sensitivity.py`: analytical Sobol indices from PCE coefficients (the "sensitivity for free" trick).
- Cross-check with Saltelli sampling on the surrogate.

Key Conceptual Notes

Cheap vs Honest Uncertainty Propagation

Both versions produce joint vector samples. The difference is *where the correlations live.*

Independent marginal sampling (cheap, fragmented). Each component fit independently, sampled independently. Joint distribution is the product of marginals → diagonal covariance. Misses correlations between parameters that exist in the data (e.g., team offense and defense are correlated; team strength and goalie skill are negatively correlated for bad-team-good-goalie scenarios).

Joint hierarchical posterior (honest). All parameters fit jointly in a single hierarchical model. Sample the whole vector at once → full covariance, possibly non-Gaussian. This is what Track B produces.

Cheap version typically miscalibrates the tails. Won't catastrophically break win probabilities but will produce wrong probabilities for extreme outcomes.

Why PCE Earns Its Keep for Hockey

For the soil project, PCE was essential because the truth model was ~1 s/eval. For the hockey team-level game model, evaluation is in microseconds — direct MCMC on the truth model is viable, surrogate not strictly needed for inference.

PCE earns its keep in hockey for **sensitivity analysis**: analytical Sobol indices directly from PCE coefficients via the variance-decomposition identity

$$\operatorname{Var}\left[\widehat{Y}\right] = \sum_{|\alpha|>0} c_\alpha^2 \mathbb{E}\left[\Psi_\alpha^2\right]$$

This gives a clean decomposition of outcome variance into contributions from each input (team xGF/60, team xGA/60, goalie GSAx, etc.) — without running Saltelli on millions of samples.

If you later upgrade to a possession-by-possession simulator that's slow to run, PCE returns as an inference-speed tool.

Sources of Uncertainty (Ranked by Sobol Later)

Three categories injected at different layers:

1. **Aleatoric (irreducible).** Game randomness — Poisson sampling in the game model. Even with perfect knowledge of λ, the puck still bounces.
2. **Parameter uncertainty.** Posterior width on team/goalie/player parameters from finite-sample inference.
3. **Model misspecification.** The Poisson form, the independence assumption between home/away goals, the missing factors (rest, travel, injuries). Hardest to quantify; partially addressed by Dixon-Coles corrections and held-out validation.

Sobol analysis at the end ranks which of these matter most for which outcomes.

Persistence & Reproducibility

- **xG model:** pickle under `output/models/` (`train_xg.py` writes the model and a `_metrics.json`). Still to add: version number and a config entry pointing to the active version.
- **Posterior samples:** ArviZ NetCDF under `output/posteriors/`. Both tracks write to the same format so downstream code is sampler-agnostic.
- **PCE surrogate:** pickle with input-range metadata and coefficient array.
- **Backtest results:** CSV under `output/backtests/` with one row per held-out game (date, teams, prediction, outcome, log-loss contribution).

Reproducibility checklist for any fit:

- Random seed logged
- Data cutoff date logged
- Model version logged
- Software versions (PyMC, NumPy, LightGBM) logged in metadata

---

Open Questions Deferred to "When We Get There"

These are real decisions but premature to make now:

- **OOP vs functional structure** for the inference layer. Decide once 2–3 modules are written and friction becomes obvious.
- **Whether to use `__init__.py` re-exports** for cleaner imports. Default to empty files; refactor later if imports get ugly.
- **OT/SO model.** Start with a flat 50/50 conditional on tied regulation. Upgrade to a goalie-quality-weighted shootout model later if it materially changes calibration.

- **Score effects, rest, travel, back-to-backs.** Real factors, but adding them prematurely complicates the base model. Document as v2 features.
- **Player-level point distributions.** Requires shift-level data and per-player rate modeling. Defer until team-level pipeline is calibrated.

Data and xG items found in Phases 1–2:

- **Redirect location drift.** Each season, redirects are recorded closer to the net (share under 10 ft: 24.8% in 2021-22 → 38.2% in 2025-26) while their goal rate falls (9.4% → 7.5%). The model overrates 2025-26 redirects by about 24%, and this accounts for most of the +2.4% total bias. Recency weighting helps only slightly. Retrain every season.
- **Arena recording bias.** Some rinks log shot locations closer or farther than others. MoneyPuck adjusts for it; this model does not. v2 refinement.
- **Timestamp artifacts.** The feed logs whole seconds, so an opponent's shot can appear 1–2 s before a shot at the other net. Affects about 0.5% of shots.
- **NST rush definition.** NST appears not to count an opponent's shot as a rush precursor. Unconfirmed at three events; a query-time filter gives NST parity if needed.
- **Rebound and rush windows.** 3 s and 4 s are NST conventions, not fitted. Test 2/3/4 s by retraining and comparing validation log loss.
- **Previous-event location for stoppage-type events** is missing for about 0.5% of shots.

Decisions Made

For future reference, the decisions locked in during planning:

- **First shipping target:** single-game win probability.
- **xG approach:** train own model, validate against MoneyPuck.
- **Uncertainty propagation:** honest version (joint hierarchical Bayesian via PyMC NUTS).
- **Build strategy:** Track A (AM MCMC) and Track B (PyMC) in parallel. Track A as integration test and reference; Track B as production.
- **Uncertainty sources to track:** all three (aleatoric, parameter, model misspec); rank with Sobol later, no pre-ranking.
- **Folder structure:** add `analysis/`, `validation/`, `pipelines/` as new top-level under `src/`. `game_model.py` at top level of `models/`.

Decided during Phases 1–2:

- **Raw tables stay raw.** Derived flags live in `shot_context`; filters are applied at query time.
- **xG training set:** unblocked shots, `period <= 4`, empty nets dropped.
- **Season roles:** test = `CURRENT_SEASON`, validation = `LAST_SEASON` (both in `config/settings.py`); the test season is graded once per change, never used to choose settings.
- **Rush in the model:** feed `last_event_type` and `time_since_last_event`; keep `is_rush` for reporting.
- **Strength state:** decoded shooter-relative (skater difference, total skaters, own goalie pulled), not the raw code.
- **Shot types grouped** into `wrist-snap` and `redirect` because the labels changed between seasons.

- **Recency weighting on** (1.0 / 1.25 / 1.5 / 2.0).
- **No recalibration step:** the remaining bias is concentrated in redirects, so a uniform shift would worsen other shot types.
- **Trained `.pkl` files stay out of git;** rebuild with `train_xg.py`.

Glossary

xG — Expected Goals. Per-shot probability of becoming a goal, summed over shots. Output of a classifier (usually gradient boosting) trained on shot features.

RAPM — Regularized Adjusted Plus-Minus. Ridge regression isolating player impact on shot/goal rates while controlling for teammates and opponents.

GSAx — Goals Saved Above Expected. $xGA - GA$ for goalies. Positive = better than average.

PDO — On-ice $SH\% + SV\%$. Mean-reverts to 100. Luck/regression indicator.

Corsi / Fenwick — Shot attempt differentials. Corsi includes blocks; Fenwick excludes them.

Game Score — Dom Luszczyszyn's weighted-stat composite (~1.0 = average game).

PCE — Polynomial Chaos Expansion. Spectral surrogate for expensive forward models. Orthogonal polynomial basis under input distribution.

Sobol indices — Variance-based sensitivity measures. S_i = fraction of output variance attributable to input i alone; S_{T_i} = total contribution including interactions.

NUTS — No-U-Turn Sampler. HMC variant. Default sampler in modern PyMC.

AM — Adaptive Metropolis. Random-walk MH with covariance estimated from burn-in.

ArviZ — Python library for Bayesian model diagnostics and storage. NetCDF-based `InferenceData` is the standard format.

Walk-forward backtest — Validation strategy where, for each held-out game in chronological order, models are refit using only prior data. Avoids look-ahead bias.

AUC — Area under the ROC curve. The chance the model gives a random goal a higher xG than a random non-goal. 0.5 is guessing; public xG models land around 0.75–0.80.

Log loss — How far predicted probabilities are from what happened. Lower is better; a proper scoring rule.

Calibration — Whether predicted probabilities match observed rates: shots given 10% should score about 10% of the time.

Gradient boosting — Many small decision trees built in sequence, each correcting the previous ones' errors (LightGBM, XGBoost). Contrast with a random forest, which averages independently built trees.

Early stopping — Stop adding trees once the validation score stops improving, to avoid overfitting.

Recency weighting — Counting recent seasons more in training so the model follows current recording habits.

Proper scoring rules — Loss functions for probabilistic predictions that are minimized in expectation by the true distribution. Log-loss, Brier, CRPS are proper. RMSE on probabilities is not.